

CS 3510 Algorithms: Closest Pair

Aaron Hillegass
Georgia Tech

Finding the closest pair

You are given a set of n points in the plane: $\{(x_1, y_1), \dots, (x_n, y_n)\}$

Find the two that are closest to each other.

Assume finding distance between two points is $O(1)$.

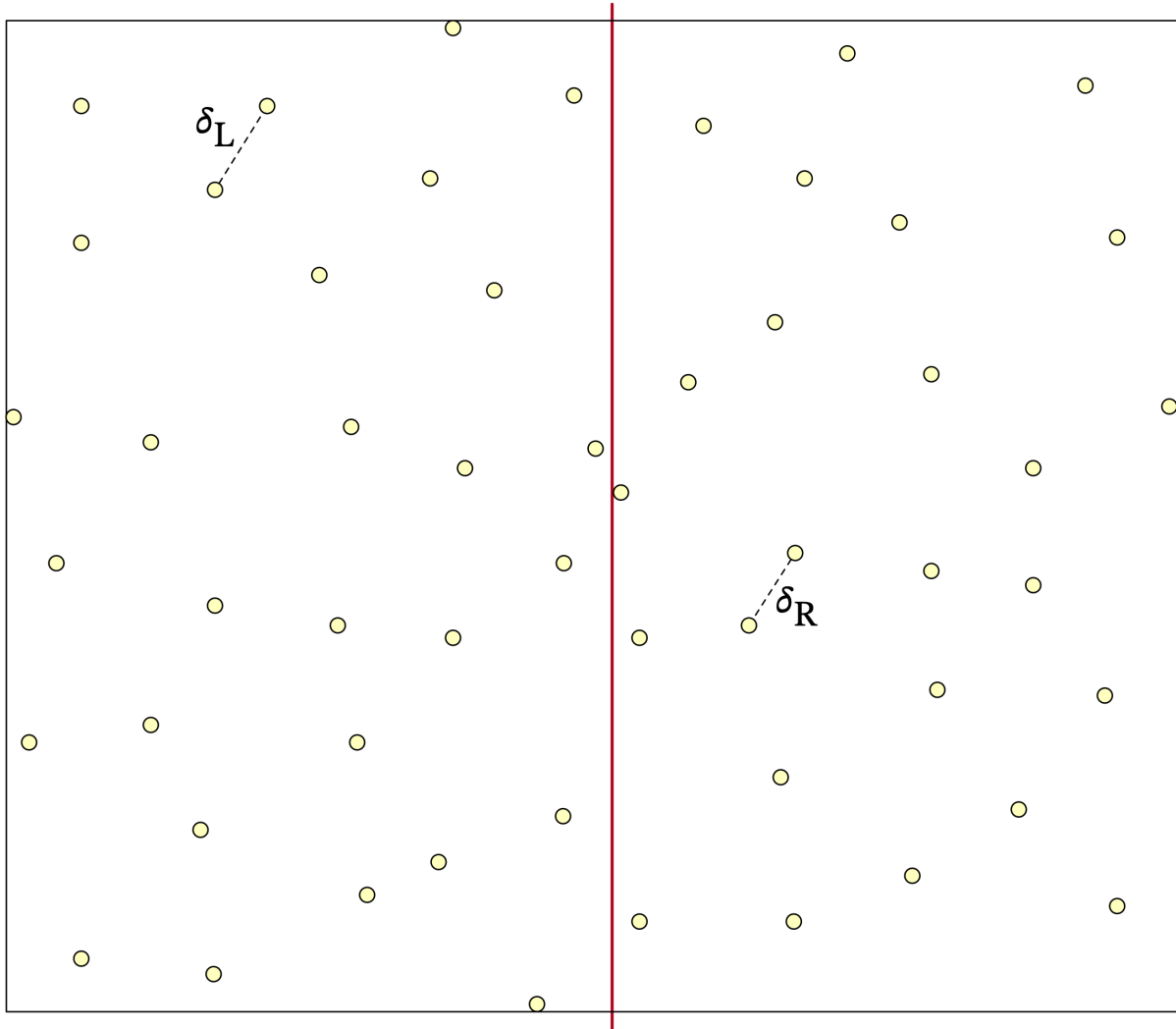
Obvious solution is $O(n^2)$. Can we do better?

Goal: $O(n \log n)$

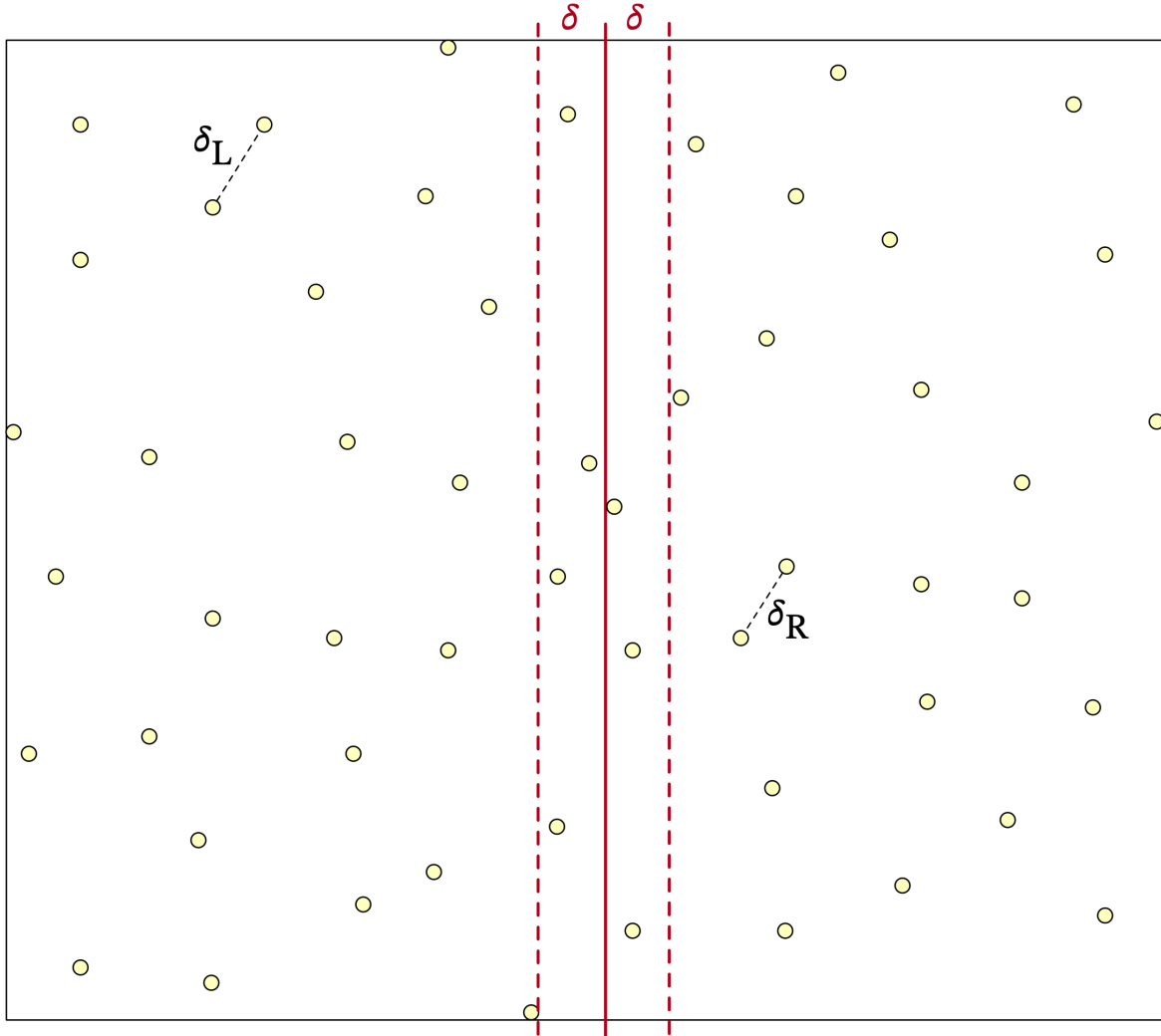
Credit: Michael Shamos, 1975

Divide?

3/17



What about pairs that straddle the divide?



So far:

```
1 define closest_with_sorted( $P$ ):  
2   if  $n == 2$ , return ( $d(P[1], P[2])$ , ( $P[1], P[2]$ ))  
3   if  $n == 3$ , return shortest_of_three( $P$ )  
4    $\text{mid} := \lfloor \frac{n}{2} \rfloor$   
5   ( $\delta_L$ , pairLeft) := closest_with_sorted( $P[1..\text{mid}]$ )  
6   ( $\delta_R$ , pairRight) := closest_with_sorted( $P[(\text{mid} + 1)..n]$ )  
7   if  $\delta_L < \delta_R$  then  $\delta := \delta_L$ , best := pairLeft  
8   else  $\delta := \delta_R$ , best := pairRight  
9    $\text{split} := (P[\text{mid}].x + P[\text{mid} + 1].x)/2$   
10  ( $\delta_S$ , pairSplit) := closest_split_pair( $P$ , split,  $\delta$ )  
11  if  $\delta_S < \delta$  then return ( $\delta_S$ , pairSplit)  
12  else return ( $\delta$ , best)
```

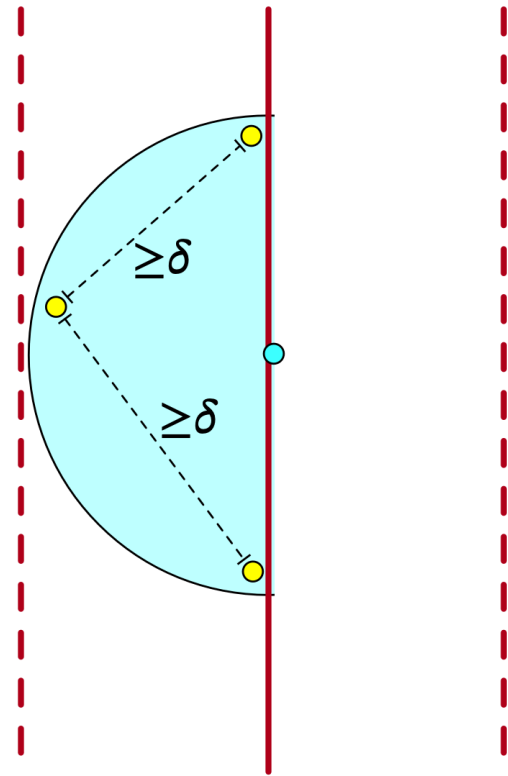
- Sort the points by x : $O(n \log n)$ once
- $T(n) = 2T(\frac{n}{2}) + O(?)$
- If we want $O(n \log n)$ we need searching the gutter to be $O(n)$

Can we limit our search in the gutter?

Facts:

- Only interested in pairs where one is on each side
- Only interested in pairs that are closer than δ
- No two on the same side can be closer than δ

Each point on the right only needs to check a max of three points on the left.

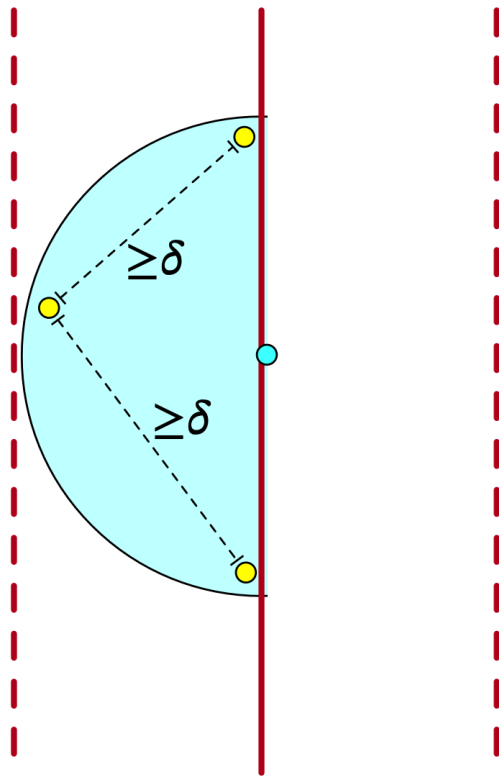


If only...

If we had a two lists, left and right) sorted by y , this would be easy an easy $O(n)$ merge-like process.

But sorting could be $O(n \log n)$! We need to keep it $O(n)$.

But...if all the points were sorted by y , we could sort them into left and right in linear time.



```
1 define closest_with_sorted2( $P_x, P_y$ ):
2   if  $n$  is 2 or 3, return dumb_version( $P_x$ )
3    $\text{mid} := \lfloor \frac{n}{2} \rfloor$ 
4    $(\delta_L, \text{pairLeft}) := \text{closest\_with\_sorted}(P_{x[1..\text{mid}]}, P_y)$ 
5    $(\delta_R, \text{pairRight}) := \text{closest\_with\_sorted}(P_{x[(\text{mid}+1)..n]}, P_y)$ 
6   if  $\delta_L < \delta_R$  then  $\delta := \delta_L$ ,  $\text{best} := \text{pairLeft}$ 
7   else  $\delta := \delta_R$ ,  $\text{best} := \text{pairRight}$ 
8    $\text{split} := (P_{x[\text{mid}]} \cdot x + P_{x[\text{mid}+1]} \cdot x) / 2$ 
9    $(\delta_S, \text{pairSplit}) := \text{closest\_split\_pair}(P_x, \text{split}, \delta, P_y)$ 
10  if  $\delta_S < \delta$  then return  $(\delta_S, \text{pairSplit})$ 
11  else return  $(\delta, \text{best})$ 
```

Searching the gutter

```
1 define closest_split_pair( $P_x$ , split,  $\delta$ ,  $P_y$ )
2    $L :=$  new List,  $R :=$  new List
3   for pair in  $P_y$ :
4     | Put pair in the right list or ignore if outside gutter
5   if either list is empty return None
6   for pair $L$  in  $L$ :
7     | Do a merge-like comparison with two below and two above in  $R$ 
8     | Note closest pair and best distance
9   return best distance and closest pair
```

- $T(n) = 2T(\frac{n}{2}) + O(n)$
- Runtime: $O(n)$

The Maximum Subarray Sum Problem

Given an array $A[1...n]$ of numbers, find a subarray with the max sum. Or find $1 \leq i \leq j \leq n$ that maximizes the sum:

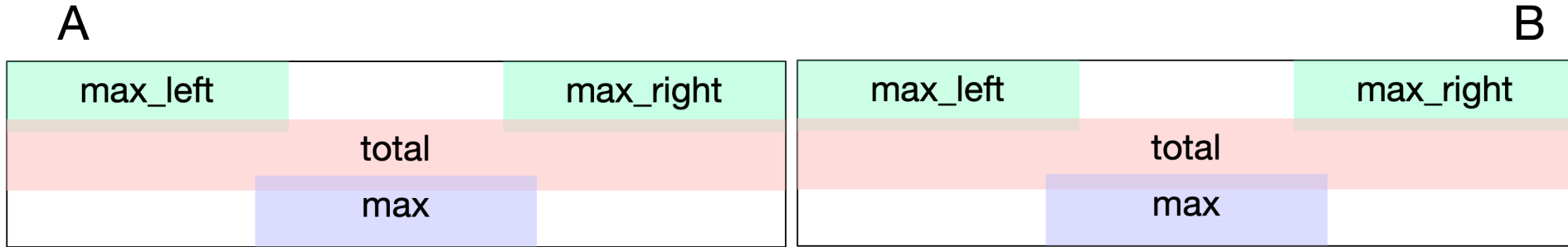
$$\sum_{k=i}^j A[k]$$

Brute force? $O(n^6)$

Grenander? $O(n^2)$

Shamos Divide-and-Conquer? $O(n \log n)$

Divide and Conquer for Max Subarray



Joining two?

- $\text{max_left} := \max(A.\text{max_left}, A.\text{total} + B.\text{max_left})$
- $\text{max_right} := \max(B.\text{max_right}, B.\text{total} + A.\text{max_right})$
- $\text{total} := A.\text{total} + B.\text{total}$
- $\text{max} := \max(\text{max_left}, \text{max_right}, A.\text{max}, B.\text{max}, A.\text{max_right} + B.\text{max_left})$

Array of one?

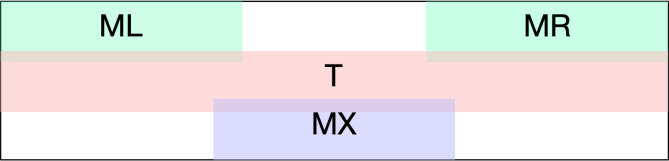
- All of the above are equal: value in the array.

Divide and Conquer for Max Subarray

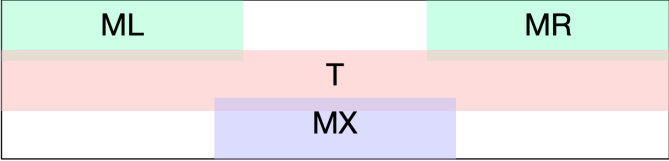
```
1 define analyze( $A$ ):
2    $n := \text{len}(A)$ 
3   if  $n == 1$ :
4     | return ( $ml : A[1], mr : A[1], \text{total} : A[1], mx : A[1]$ )
5    $c := \lfloor \frac{n}{2} \rfloor$ 
6    $L := A[1..c], R := A[(c + 1)..n]$ 
7   ( $lml, lmr, lttotal, lmx := \text{analyze}(L)$ )
8   ( $rml, rmr, rttotal, rmx := \text{analyze}(R)$ )
9   return ( $ml : \max(lml, lttotal + rml),$ 
10    |  $mr : \max(rmr, rttotal + lmr),$ 
11    |  $\text{total} : lttotal + rttotal,$ 
12    |  $mx : \max(lml, rml, lmx, rmx, lrm + rlm))$ 
```

```
1 define max_subsum( $A$ ):
2   | ( $_, _, _, mx$ ) := analyze( $A$ )
3   return  $mx$ 
```

A



B



-8	5	-2	-1	4	3	-7	-6	2	2	2	-3	-2	-1	4	3
----	---	----	----	---	---	----	----	---	---	---	----	----	----	---	---

ML:-3	ML:-2	ML:7	ML:-7	ML:4	ML:2	ML:-2	ML:7
MR:5	MR:-1	MR:7	MR:-6	MR:4	MR:-1	MR:-1	MR:7
T:-3	T:-3	T:7	T:-13	T:4	T:-1	T:-3	T:7
MX:5	MX:-1	MX:7	MX:-6	MX:4	MX:2	MX:-1	MX:7

ML:-3	ML:7	ML:6	ML:4
MR:2	MR:-6	MR:3	MR:7
T:-6	T:-6	T:3	T:4
MX:5	MX:7	MX:6	MX:7

ML:1	ML:7
MR:-4	MR:7
T:-12	T:7
MX:9	MX:7

ML:1
MR:7
T:-5
MX:9

Divide and Conquer for Max Subarray

$$T(n) = 2T\left(\frac{n}{2}\right) + O(1)$$

$$O(n)$$

Questions?

Slides by Aaron Hillegass